DEFENSIVE VAULT ARCHITECTURE v1.0

Zero-Trust Object Custody, Release, Recovery, and Proof Architecture

Foundational rule
The vault is never broadly unlocked or mounted. Every protected object follows: Verify -> Authorize -> Release -> Zeroize.

1. Purpose and Security Model

This vault is a defensive custody system for sensitive tools and artifacts. It is designed as a governed security state machine rather than a conventional encrypted folder. Storage, identity, admission, release, recovery, backup, revocation, migration, and retirement are treated as explicit state transitions with evidence, authorization, and audit requirements.

2. Root of Trust and Identity

The vault root is hardware anchored using TPM 2.0 or equivalent secure hardware. Platform measurements, secure boot state, firmware measurements, and sealed key material form the host trust basis.

The USB token is a possession factor and activation share, not the vault identity. It may carry a short-lived activation share or signed challenge-response credential.

Operator knowledge is a separate factor derived using a hardened password KDF such as Argon2id or a high-iteration PBKDF2 configuration.

Vault identity is bound to the TPM-sealed root, monotonic state, audit epoch, and token-binding keys. Identity must resist cloning, rollback, and unauthorized forks.

3. Monotonic State and Anti-Rollback

Every security-critical state transition increments a trusted monotonic value, preferably a TPM NV counter or equivalent secure element counter. Critical metadata, policy versions, revocation lists, audit checkpoints, invariant roots, and backup state are cryptographically bound to that value. A restored state with a stale counter, invalid signature, or inconsistent root is rejected.

4. Compartmentalized Key Hierarchy

Layer	Purpose
Hardware-sealed root	Minimal-use root material; never exposed to clients.
Vault wrapping key	Wraps narrower operational keys.
Per-object key	Encrypts one protected object or record.

Layer	Purpose
Audit key	Signs audit checkpoints and event proofs.
Recovery key/share	Supports quorum-based recovery only.
Token-binding key	Binds possession credentials to the current vault epoch.
Session/release key	Ephemeral key material for a single authorized release.

5. Object-Level Release Protocol

1. Client submits an authenticated request naming a single opaque object identifier, purpose, and intended consumer.

2. Vault validates identity, token state, attestation, policy freshness, audit readiness, time trust, invariant state, and capability authorization.

3. Vault decrypts only the requested object into locked, non-swappable memory.

4. Data crosses a minimal local IPC conduit with strict peer authentication and no master-key exposure.

5. The release buffer is explicitly zeroized immediately after handoff.

6. The release produces a signed audit event and advances any required state counters.

6. Metadata Protection and Admission

The vault maintains an encrypted authenticated index and exposes only opaque, randomized references externally. Sensitive filenames, timestamps, categories, provenance fields, and structural metadata remain encrypted. Payload padding may be used to reduce size leakage. Admission is a verification pipeline, not a write operation: provenance, hashes, signatures, version, SBOM where available, and explicit approval are recorded before an object enters trusted custody.

7. Isolation and Memory Hygiene

The vault runs as a minimal dedicated service with reduced privileges, no network access by default, strict syscall and namespace restrictions, and a narrow audited IPC surface.

Decrypted content and operational keys reside only in locked memory for the minimum required duration. Sensitive buffers are explicitly zeroized using mechanisms the compiler cannot optimize away.

Crash dumps, shell history, temporary files, swap, telemetry, debug output, and logs must never receive plaintext tool content or secret key material.

8. Tamper Response and Formal Deny

Condition	Required response
Unexpected token removal	Seal current release path and terminate active authorization.
Attestation mismatch	Seal; invalidate active authorization; require re-attestation.
Repeated authentication failures	Escalate to governed lockout or recovery policy.

Condition	Required response
Audit unavailable / capacity exhausted	Fail closed for sensitive operations.
Software integrity violation	Revoke affected epoch and require re-provisioning or governed recovery.
Ambiguous device, clock, policy, or invariant state	Deny operation. No best-effort fallback.

9. Recovery and Backup

Recovery is a security epoch transition, not an administrative bypass. It requires threshold approval, platform attestation, explicit witness evidence, rotation of affected key material, invalidation of prior token bindings, and a distinct signed audit trail. Recovery never reveals the original master key.

Backups are continuations of vault state rather than ordinary snapshots. They include monotonic state, audit checkpoints, revocation history, invariant state, provenance, and cryptographic bindings. Restoration is itself a governed transition and produces a new auditable epoch.

10. Live Invariant Graph

Policy, revocation, audit, key hierarchy, attestation, identity, recovery status, and state counters form a mutually dependent invariant graph. A change in one domain automatically invalidates or recomputes dependent invariants. The vault refuses sensitive operation when the graph cannot be recomputed consistently.

11. Vault Identity Continuity Protocol

The vault possesses a logical identity distinct from any one motherboard, TPM, or storage device. Hardware replacement or migration is a governed IDENTITY_MIGRATION event. It requires old-root evidence where available, recovery quorum, attestation of the new platform, external witness evidence, generation of a new hardware binding, and continuation of the logical identity through a new security epoch.

12. Security Resource Budget

The architecture defines explicit operating envelopes for release latency, authentication failures, audit capacity, attestation freshness, release frequency, cryptographic lifetime, recovery frequency, and reserved security resources. Crossing a security-critical boundary produces a governed degraded or sealed state rather than silent relaxation of controls.

13. Slow-Compromise and Drift Detection

The vault maintains behavioral baselines for releases, denials, authentication failures, policy changes, recovery attempts, administrative actions, attestation measurements, and security exceptions. Significant deviation can raise DRIFT_SUSPECTED, increasing verification requirements or sealing the vault according to policy. Drift evidence is retained and independently verifiable.

14. Independent Verification Plane

The runtime is not the sole judge of its correctness. An independently implemented verifier checks audit continuity, monotonic state, policy hashes, invariant roots, identity epochs, revocation state, and attestation evidence. Runtime/verifier disagreement is treated as INTERNAL_CONTRADICTION and forces the vault into a sealed or degraded state pending governed resolution.

15. Constitutional Authority Ceiling

The vault has a permanent authority ceiling enforced as structural invariants. It may not grant itself authority, rewrite its own constitutional invariants, approve its own recovery, suppress or rewrite audit evidence, weaken required authentication factors, silently broaden capability scope, or transform storage authority into execution authority.

16. Vault Federation Protocol

Independent vaults do not inherit trust from one another. Federation begins with mutual attestation and signed manifest exchange. Object transfer requires source provenance, destination admission approval, compatible policy and security epochs, explicit capability authorization, and signed receipts from both sides. Federation never creates a universal trust domain.

17. Invariant Constitution

Every invariant has an identifier, normative definition, dependency set, version, tests, migration rules, cryptographic digest, and governance threshold. Invariant changes require a higher authorization threshold than ordinary policy changes. The vault therefore proves not only that invariants passed, but exactly which definition of correctness was

evaluated.

18. Assumption Register and Proof Provenance

Every security proof declares the assumptions on which it depends. Examples include TPM integrity, cryptographic algorithm strength, random number generation, trusted time, firmware measurements, compiler/toolchain integrity, operator quorum, external witness availability, and verifier correctness.

Assumption record must contain	Purpose
Identifier + version	Stable reference for proof dependencies.
Supporting evidence	Why the assumption is currently accepted.
Confidence / validity state	Current architectural status.
Revalidation trigger	Event or time condition that forces reassessment.
Dependent proofs and invariants	Enables automatic impact propagation.
Failure action	Seal, degrade, revoke, migrate, or recover.

Propagation rule: Evidence changes -> assumption invalidated -> dependent proofs invalidated -> invariants recomputed -> affected capabilities revoked -> governed state transition -> signed audit evidence.

19. Canonical State Machine

Primary path	`UNINITIALIZED -> PROVISIONING -> SEALED -> VERIFYING -> AUTHORIZED_RELEASE -> ZEROIZING -> SEALED`
Exceptional states	`DEGRADED | DRIFT_SUSPECTED | TAMPERED | RECOVERY | MIGRATION | REVOKED | RETIRED`

There is no persistent UNLOCKED state and no ordinary mounted-vault state. Authorization applies to a single governed release, after which plaintext and ephemeral authorization material are destroyed.

20. Production Proof Obligations

• Token clone resistance demonstrated against realistic descriptor and media cloning.

• Lawful recovery demonstrated without disclosure of the original root secret.

• Zero plaintext residue demonstrated across RAM lifecycle, temp storage, swap, dumps, logs, and failure paths.

• Tamper-evident audit continuity demonstrated across rotation, capacity pressure, restore, migration, and recovery.

• Storage/execution separation demonstrated structurally and by test; the vault contains no execution authority.

• Rollback resistance demonstrated for metadata, policy, revocation state, backups, audit epochs, and invariant definitions.

• Independent verifier disagreement demonstrated to fail closed.

• Assumption invalidation demonstrated to propagate to every dependent proof, invariant, and capability.

21. Freeze Criteria for Architecture v1.0

The architecture may be frozen when the state machine, identity model, key hierarchy, recovery model, audit model, invariant constitution, authority ceiling, proof obligations, and assumption register are stable enough that implementation work no longer requires redefining the trust model. Subsequent changes must be versioned architectural amendments rather than silent reinterpretations.

Final invariant

No security claim may possess greater certainty than the evidence supporting its least-certain dependency.